# Studio Nx

Backend(FastAPI/Python) 33 — POST /api/v1/auth/refresh

2026-10-12

# Backend(FastAPI/Python) 33 — 라우터 `POST /api/v1/auth/refresh`

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/api/v1/auth.py` (함수 이름 `refresh`) |
| 작업 | P1-05 로그인: Access Token 재발급 (PRD 01 FR-105, FR-108) |
| 작성 | Claude (AI), 2026-10-07, 사용자 검토 |
| 이전 · 다음 | 32 `POST /auth/login` · 34 `POST /auth/logout` |

## 1. 하는 일

브라우저에 남아 있는 **리프레시 쿠키**로 새 Access Token(30분)을 줍니다. 요청 본문은 없으며, 쿠키가 자동으로 실려 옵니다.

```
POST /api/v1/auth/refresh
Cookie: snx_refresh=eyJ…

→ 200 {"access_token": "eyJ…(새 30분)", "token_type": "bearer", "expires_in": 1800}
```

| 쿠키 | 응답 | 화면 (`api.js`) |
|---|---|---|
| 정상 (3일 안, 활동 회원) | 200과 새 Access Token | 메모리에 저장하고, 하던 요청을 계속합니다 |
| **없음** (로그인하지 않은 방문자) | 401 "Please sign in." | 로그인하지 않은 상태로 처리합니다 (정상적인 흐름) |
| 만료, 위조, Access Token, 엉터리 값 | 401 "Please sign in again." | 로그인 화면으로 보냅니다 |
| 진짜지만 회원이 탈퇴했거나 정지됨 | 401 "Please sign in again." | 로그인 화면으로 보냅니다 |

## 2. 언제 쓰나

P1-02의 `refreshAccessToken()`이 부릅니다. **이 라우터가 생기면서 P1-02의 재발급이 실제로 동작하게 되었습니다.**

| 상황 | 부르는 쪽 |
|---|---|
| 페이지를 새로 열었을 때 (메모리의 토큰이 비어 있음) | `apiFetch`가 요청 전에 먼저 부릅니다 (JS 02) |
| 30분이 지나 API가 401로 응답할 때 | `apiFetch`가 재발급을 받은 뒤 같은 요청을 한 번 다시 보냅니다 |
| 헤더 버튼을 그릴 때, 로그인 전용 페이지를 열 때 | `isLoggedIn()`과 `requireLogin()` (JS 06, 07) |

**ai-note 34(P1-02, 03 실제 사이트 시험)에서 `/auth/refresh → 500`이었던 결과**가 이제 200과 401로 바뀝니다.

## 3. 코드

**import** (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status
```

```
from pydantic import EmailStr

from app.api.deps import Db
from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
from app.core.security import create_access_token, create_refresh_token, decode_refresh_token
from app.schemas.auth import (…, TokenResponse)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import입니다. **`AppError`, `ErrorCode`, `decode_refresh_token`을 이번에 추가했습니다.** 이 함수는 `Annotated`, `Cookie`, `Response`, `Db`, `get_settings`, `AppError`, `ErrorCode`, `create_access_token`, `decode_refresh_token`, `TokenResponse`, `auth_service`를 사용합니다.

**함수**

```
@router.post(
    "/refresh",
    response_model=TokenResponse,
    responses={401: {"model": ErrorResponse}},
)
def refresh(
    response: Response,
    db: Db,
    refresh_token: Annotated[str | None, Cookie(alias=REFRESH_COOKIE)] = None,
) -> TokenResponse:
    if refresh_token is None:
        raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in.")
    user_id = decode_refresh_token(refresh_token)
    user = auth_service.get_active_user(db, user_id)
    return TokenResponse(
        access_token=create_access_token(user.user_id),
        expires_in=get_settings().jwt_access_minutes * 60,
    )
```

함수 머리는 템플릿에 있었고, 몸통을 채웠습니다.

## 문법 풀이

| import 문법 | 뜻 |
|---|---|
| `from fastapi import …, Cookie, …` | 요청의 쿠키에서 값을 꺼낸다는 표시입니다 (템플릿에 이미 있었습니다) |

| 식 | 뜻 |
|---|---|
| `Annotated[str \| None, Cookie(alias="snx_refresh")]` | 쿠키 `snx_refresh`의 값을 `refresh_token` 인자로 받습니다. 파이썬 이름과 쿠키 이름이 달라서 `alias`로 연결합니다 |
| `= None` | 쿠키가 없으면 `None`입니다 (오류가 아닙니다) |
| `response: Response` | 지금은 쓰지 않습니다. 나중에 리프레시 토큰을 새로 갈아 끼울 때(아래 §5) 쿠키를 심는 데 씁니다 |

# 4. 흐름

1. 쿠키가 없으면 401 "Please sign in."으로 응답합니다.

2. 28번으로 토큰을 검사해서 회원 번호를 얻습니다 (실패하면 401 "Please sign in again.").
3. 30번으로 회원이 아직 활동 중인지 확인합니다 (아니면 401).
4. 26번으로 새 Access Token을 만들어 200으로 응답합니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 쿠키가 없을 때와 토큰이 이상할 때의 문구를 다르게 함 | 쿠키가 없는 것은 "처음부터 로그인하지 않았다"는 뜻이라 정상입니다. 쿠키가 있는데 안 되는 것은 "로그인이 풀렸다"는 뜻입니다. 코드는 둘 다 `UNAUTHORIZED`이므로 `api.js`의 처리는 같습니다 (06번과 같은 구분) |
| 토큰 검사(28) 다음에 회원 확인(30) | 토큰은 진짜여도 그사이 회원이 탈퇴하거나 정지되었을 수 있습니다 (30 §2) |
| 리프레시 토큰을 갈아 끼우지 않음 (템플릿의 선택 사항) | 갈아 끼우면 재발급할 때마다 3일이 다시 늘어나서 사실상 영원히 로그인된 상태가 됩니다. 또 체크 여부(`remember`)를 토큰에 따로 담아야 합니다. 지금은 **로그인하고 3일이 지나면 다시 로그인**하는 단순한 방식으로 둡니다 |
| 실패해도 쿠키를 지우지 않음 | 오류 응답은 01번 처리기가 새로 만들기 때문에, 여기서 붙인 쿠키가 실리지 않습니다. 쿠키는 3일이 지나면 브라우저가 지우고, 로그아웃(34)할 때도 지웁니다 |

## 6. 시험 (`tests/test_auth_routes.py`에 6개를 추가해서 모두 40개, 실제 앱과 MySQL로 실행)

시험 도구(`TestClient`)는 브라우저처럼 로그인 응답의 쿠키를 기억했다가 다음 요청에 실어 보냅니다.

| 시험 | 확인한 것 |
|---|---|
| 로그인 후 재발급 (체크함, 체크하지 않음) | 200, `bearer`, 1800이고, 새 토큰을 05번으로 읽으면 그 회원 번호가 나옵니다 |
| 쿠키 없음 | 401 "Please sign in." |
| 엉터리 쿠키, Access Token을 쿠키에 넣음 | 401 `UNAUTHORIZED` "Please sign in again." |
| 로그인 뒤 탈퇴 | 쿠키가 남아 있어도 401입니다 (30번이 막습니다) |

전체 시험 231개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.